Incremental Card-Based Roguelite: Mechanics, Balance, and Implementation

I. Universal Modifier Data Model and System Architecture

The fundamental architecture of a highly scalable, systems-driven hybrid card game rests upon a singular, unified data model. By reducing all interactive game entities—characters, hazards, environmental effects, and outpost upgrades—to a shared quantitative space, the engine bypasses the need for bespoke logic handlers or hardcoded edge cases. Instead, it relies on a universal modifier system where interactions are defined as arrays of operational tuples.[1] This paradigm borrows heavily from the structural data models seen in reverse-dungeon simulators, where traps and units share underlying stat modifications.[3]

The Five-Stat Space Paradigm

The quantitative boundaries of the game are strictly confined to five integers. By severely limiting the stat space, the complexity emerges from the interactions rather than the sheer volume of variables.

1. **HP (Health Points):** The ultimate survival metric. Reaching zero triggers a loss state for the entity. In the context of hazards, HP reduction represents physical attrition.
2. **Power:** The base scalar for offensive output. Power is a dynamic variable that acts as the starting integer for any card or hazard that inflicts HP reduction on an opposing entity.
3. **Speed:** Determines turn-order frequency and action economy priority. This stat is critical for pacing and governs the continuous timeline algorithm.
4. **Defense:** The primary mitigation scalar. It operates primarily against incoming flat modifications to HP, serving as a subtractive buffer before HP damage is finalized.
5. **Energy:** The turn-specific resource budget. Because the entire hand of cards is drawn and visible each turn, Energy serves as the sole economic bottleneck restricting action volume.

The Modifier Mechanics and Resolution Order

All changes to these stats are processed as a Modifier. To maintain mathematical stability and prevent edge-case computational collapse, the engine enforces a strict resolution order for all active modifiers: Base Value $\rightarrow$ Flat Modification $\rightarrow$ Percentage Modification $\rightarrow$ Multiplicative Scaling.[5]

For example, an entity with a base Power of 10, a flat addition of +5, and a percentage

modification of +50% will resolve as (10 + 5) * 1.5 = 22.5 (rounded to 22). By ensuring flat modifications are calculated before percentage scalars, the game avoids the pitfall of percentage buffs yielding zero impact on base-zero stats.

JSON Schema Architecture

To ensure seamless consumption by both the React frontend and the Python simulation backend, the schema relies on a rigid hierarchical JSON structure.[4] This data-driven approach allows non-programmers to define complex interactions entirely within text files.[1]

JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "definitions": {
  "Modifier": {
   "type": "object",
   "properties": {
    "stat": { "enum": },
    "operation": { "enum": },
    "value": { "type": "number" },
    "duration": { "type": "integer", "description": "0 = instant, -1 = permanent, >0 = turns" },
    "target": { "enum": }
   },
   "required": ["stat", "operation", "value", "duration", "target"]
  }
 },
 "type": "object",
 "properties": {
  "Entity": {
   "type": "object",
   "properties": {
    "id": { "type": "string" },
    "type": { "enum": },
    "base_stats": {
     "type": "object",
     "properties": {
      "HP": { "type": "number" }, "Power": { "type": "number" },
      "Speed": { "type": "number" }, "Defense": { "type": "number" },
      "Energy": { "type": "number" }
```

```
        }
      },
      "modifiers": { "type": "array", "items": { "$ref": "#/definitions/Modifier" } }
    }
  }
 }
}
```

Instantiation Examples: Cards, Characters, and Regions

The universal data model allows a massive variety of tactical implementations. A hazard is simply an entity that plays a card on the player automatically. An outpost upgrade is simply a modifier with a duration of -1 (permanent) applied to the base stats of the roster.[10]

Table 1: Example Cards (Universal Modifier Implementation)

Card Name	Energy Cost	Target	Stat	Operation	Value	Duration	Functional Result
Arcane Strike	2	ENEMY_SINGLE	HP	FLAT_SUB	15	0	Deals 15 direct damage to a single enemy.
Immolate	1	ENEMY_SINGLE	HP	FLAT_SUB	4	3	Deals 4 damage immediately, and 4 damage for the next 3 turns (Damage over time).

Shield Bash	2	ENEMY_SINGLE	HP	FLAT_SUB	8	0	Deals 8 direct damage.
Shield Bash (Stun)	0	ENEMY_SINGLE	Speed	PCT_SUB	100	1	Secondary array effect: Removes the target's next turn by reducing speed generation to zero.
Sweeping Blade	3	ENEMY_ALL	HP	FLAT_SUB	12	0	Deals 12 damage to all enemies simultaneously.
Phalanx	2	ALLY_SINGLE	Defense	FLAT_ADD	15	2	Grants 15 Defense for 2 turns, heavily mitigating incoming flat damage.

Adrenaline	1	ALLY_SINGLE	Speed	PCT_ADD	30	3	Increases Speed by 30% for 3 turns, pushing the unit up the turn order.
Cleanse	1	ALLY_SINGLE	HP	FLAT_ADD	10	0	Restores 10 HP. (A secondary tag clears negative active modifiers).
Deep Focus	0	SELF	Energy	FLAT_ADD	3	0	Restores 3 Energy instantly, acting as an economic battery.
Acid Flask	1	ENEMY_ALL	Defense	PCT_SUB	25	2	Reduces all enemy Defense by 25% for 2 turns,

							amplifying subsequent flat attacks.
Tripwire (Hazard)	0	ALLY_SINGLE	Power	PCT_SUB	50	2	Hazard effect: Reduces the target's Power by 50% for 2 turns.
Miasma (Hazard)	0	ALLY_ALL	HP	FLAT_SUB	2	-1	Hazard effect: Permanently reduces HP by 2 every turn for the remainder of the encounter.

Characters are defined by their baseline integer arrays and their innate passives, which are simply pre-loaded modifiers that are injected into the combat resolver at the start of any encounter.

Table 2: Procedurally Generated Characters

Character Name	Base Stats (HP / Pwr / Spd / Def / Ene)	Innate Passive Modifier Array (Permanent

		Duration)
Vanguard Sentinel	140 / 12 / 80 / 20 / 3	``
Ember Mage	65 / 28 / 115 / 5 / 4	``
Field Tactician	85 / 15 / 105 / 12 / 5	``

Regions define the macro-structure of the run. Each region guarantees a rigid narrative progression (Approach $\rightarrow$ Settlement $\rightarrow$ Stronghold), ensuring a predictable pacing curve while randomizing the exact modifiers.[11]

Table 3: Region and Encounter Chains

Region Name	Encounter 1 (Approach)	Encounter 2 (Settlement)	Encounter 3 (Stronghold)	Region Meta-Modifier (Clear Reward)
The Ashen Wastes	Hazard: Ash Storm (-15% Speed)	Combat: Scavenger Patrol	Combat: Warlord Vanguard	``
Whispering Thicket	Event: Lost Caravan (Trade HP for Pwr)	Hazard: Toxic Spores (-HP over time)	Combat: Fungal Behemoth	``

II. Single-Resolver Combat Architecture

To unify interactions across disparate game phases, the engine processes player cards, hazards, and narrative events through a singular ResolverEngine.[10] Turn order is calculated using a continuous Charge Time (CT) mechanic inspired by classic tactical RPGs, where speed dictates turn frequency and dictates priority resolution.[15]

Turn Order Mathematics and the Charge Time System

Rather than utilizing a static round-robin turn system, each entity possesses a continuous

variable, CT. At every computational tick t, the entity's Speed is added to its CT:

$$CT_{i,t+1} = CT_{i,t} + Speed_i$$

When $CT_i \geq 100$, the entity is granted an active turn. If multiple entities cross the threshold simultaneously, the highest CT overflow takes priority. After an action phase concludes, CT_i is decremented by 100. This mathematical framework allows a character with 150 Speed to take exactly three turns in the time it takes a character with 50 Speed to take one. Speed becomes a highly volatile and valuable resource.

Energy Refresh and Duration Tracking

Energy is a volatile resource that completely resets at the beginning of an entity's turn based on their Base Energy plus any permanent modifiers. It does not carry over between turns unless a specific modifier explicitly adds to the base limit.

Effect duration is strictly tracked in turns. When an entity begins its turn, the ResolverEngine iterates through all active modifiers attached to that entity. Any modifier with a duration > 0 is decremented by 1. Modifiers reaching 0 are purged from the array before stats are recalculated. Modifiers with a duration of -1 (permanent) bypass this decrement loop.

Combat Resolver Pseudocode (Python Backend)

The following implementation targets the Python backend for balance simulation. It handles all three encounter types (combat, hazard, event) by simply passing the appropriate arrays into the engine.[16]

Python

```python
import copy

class Modifier:
    def __init__(self, target, stat, operation, value, duration):
        self.target = target
        self.stat = stat
        self.operation = operation
        self.value = value
        self.duration = duration # 0 = instant, -1 = permanent, >0 = turns
```

```python
class Entity:
    def __init__(self, entity_id, stats, faction):
        self.id = entity_id
        self.base_stats = copy.deepcopy(stats)
        self.current_stats = copy.deepcopy(stats)
        self.active_modifiers =
        self.ct = 0
        self.faction = faction # "PARTY" or "ENEMY"

class ResolverEngine:
    def __init__(self, party, enemies):
        self.party = party
        self.enemies = enemies
        self.active_entities = party + enemies

    def apply_modifier(self, source, modifier):
        targets = self._resolve_targets(source, modifier.target)
        for target in targets:
            if modifier.duration == 0:
                # Instant resolution (e.g., direct flat damage or direct flat heal)
                self._calculate_instant_effect(target, modifier)
            else:
                # Add to active modifier stack for ongoing evaluation
                target.active_modifiers.append(copy.deepcopy(modifier))

        self._recalculate_all_stats()

    def _resolve_targets(self, source, target_enum):
        if target_enum == "SELF":
            return [source]
        elif target_enum == "ALLY_SINGLE":
            # In actual implementation, requires player input selection
            return [t for t in self.active_entities if t.faction == source.faction][:1]
        elif target_enum == "ALLY_ALL":
            return [t for t in self.active_entities if t.faction == source.faction]
        elif target_enum == "ENEMY_SINGLE":
            # Requires player input selection
            return [t for t in self.active_entities if t.faction!= source.faction][:1]
        elif target_enum == "ENEMY_ALL":
            return [t for t in self.active_entities if t.faction!= source.faction]
        elif target_enum == "GLOBAL":
            return self.active_entities
```

```
        return

    def _calculate_instant_effect(self, target, modifier):
        # Mitigation logic using target's current Defense against FLAT_SUB HP
        if modifier.stat == "HP" and modifier.operation == "FLAT_SUB":
            mitigated_value = max(1, modifier.value - target.current_stats)
            target.current_stats["HP"] -= mitigated_value
        elif modifier.stat == "HP" and modifier.operation == "FLAT_ADD":
            target.current_stats["HP"] += modifier.value
        elif modifier.stat == "Energy" and modifier.operation == "FLAT_ADD":
             target.current_stats["Energy"] += modifier.value

    def _recalculate_all_stats(self):
        # Operations priority map: FLAT operations resolve before PCT operations
        priority = {"FLAT_ADD": 1, "FLAT_SUB": 1, "PCT_ADD": 2, "PCT_SUB": 2, "MULTIPLY": 3}

        for entity in self.active_entities:
            # Preserve current HP and Energy; reset Power, Speed, Defense to Base
            current_hp = entity.current_stats["HP"]
            current_energy = entity.current_stats["Energy"]
            entity.current_stats = copy.deepcopy(entity.base_stats)
            entity.current_stats["HP"] = current_hp
            entity.current_stats["Energy"] = current_energy

            # Sort modifiers to ensure math stability
            sorted_mods = sorted(entity.active_modifiers, key=lambda x: priority.get(x.operation, 99))

            for mod in sorted_mods:
                if mod.stat in ["HP", "Energy"] and mod.duration > 0:
                    # Over-time HP/Energy changes are handled at the start of the turn, not recalculated
statically
                    continue

                if mod.operation == "FLAT_ADD":
                    entity.current_stats[mod.stat] += mod.value
                elif mod.operation == "FLAT_SUB":
                    entity.current_stats[mod.stat] = max(0, entity.current_stats[mod.stat] - mod.value)
                elif mod.operation == "PCT_ADD":
                    entity.current_stats[mod.stat] *= (1 + (mod.value / 100.0))
                elif mod.operation == "PCT_SUB":
                    entity.current_stats[mod.stat] *= max(0, (1 - (mod.value / 100.0)))

    def tick(self):
```

```
    # Main combat loop
    while True:
        # Check terminal state conditions
        if all(p.current_stats["HP"] <= 0 for p in self.party): return "LOSS"
        if all(e.current_stats["HP"] <= 0 for e in self.enemies): return "WIN"

        # Advance time
        for entity in self.active_entities:
            if entity.current_stats["HP"] > 0:
                entity.ct += entity.current_stats

        # Filter entities that have crossed the action threshold
        ready_entities = [e for e in self.active_entities if e.ct >= 100 and e.current_stats["HP"] > 0]
        if not ready_entities:
            continue

        # Sort by highest CT overflow to handle simultaneous thresholds
        ready_entities.sort(key=lambda x: x.ct, reverse=True)
        active_entity = ready_entities

        self._execute_turn(active_entity)

def _execute_turn(self, entity):
    # 1. Process over-time effects (Duration > 0)
    mods_to_remove =
    for mod in entity.active_modifiers:
        if mod.duration > 0:
            # Apply DoT or HoT
            if mod.stat == "HP" and mod.operation == "FLAT_SUB":
                entity.current_stats["HP"] -= max(1, mod.value - entity.current_stats)
            elif mod.stat == "HP" and mod.operation == "FLAT_ADD":
                entity.current_stats["HP"] += mod.value

            mod.duration -= 1
            if mod.duration == 0:
                mods_to_remove.append(mod)

    # Clean up expired modifiers
    for mod in mods_to_remove:
        entity.active_modifiers.remove(mod)

    self._recalculate_all_stats()
```

```
        # Check death from DoT before taking action
        if entity.current_stats["HP"] <= 0:
            return

        # 2. Reset Energy to current calculated maximum
        entity.current_stats["Energy"] = entity.base_stats["Energy"]

        # 3. Request Action (from AI or Player)
        action_modifiers = self._get_action(entity)
        for mod in action_modifiers:
            self.apply_modifier(entity, mod)

        # 4. Consume CT
        entity.ct -= 100

    def _get_action(self, entity):
        # Placeholder for AI logic or React UI asynchronous input
        # Returns a list of Modifier objects generated by a selected Card
        return
```

Because Hazards and Events process logic identically to Combat, an event like traversing a toxic swamp simply queues an ENEMY_ALL modifier with a -HP over time into the ResolverEngine. The engine ticks until the duration expires, evaluating damage exactly as it would an enemy's poison attack.

III. Card Upgrade Pathways and Meaningful Choices

Progression in this incremental roguelite relies on horizontal tactical expansion rather than pure vertical numerical scaling. Utilizing a model successfully demonstrated by titles such as *Across the Obelisk*, card upgrades append new secondary modifiers to the base arrays rather than simply overwriting base damage integers.[19] This design framework prevents early-game cards from becoming mathematically obsolete in the late game, while simultaneously avoiding the exponential power creep associated with idle-game scaling.[20]

By confining upgrades to a rigid 3-tier system, branching pathways compel the player to adapt their deck architecture to contextual campaign needs. If a player is facing a Stronghold known for high-defense elites, they will naturally prioritize upgrade branches that append Defense PCT_SUB modifiers over branches that merely increase raw flat power.

To ensure meaningful choice, upgrade branches must adhere to an exclusionary rule: if Branch A provides economic manipulation (e.g., Energy cost reduction), Branch B must provide a comparable mechanical subversion (e.g., altering targeting rules or appending potent debuffs).

Table 4: Card Upgrade Branching Trees

Base Card	Tier 1 Upgrade Choices	Tier 2 Upgrade Choices	Tier 3 (Capstone) Upgrade Choices
Strike ``	**1A:** Add `` **1B:** Reduce Energy Cost to 0	**2A (from 1A):** Increase Def Shred Dur to 4 **2B (from 1A):** Add ``	**3A:** Target becomes ENEMY_ALL (AoE) **3B:** Base damage value increased to 25
Shield Wall ``	**1A:** Target becomes ALLY_ALL **1B:** Add `` (Instant Heal)	**2A (from 1A):** Add `` **2B (from 1A):** Increase Dur to 4	**3A:** Modifier Dur becomes -1 (Permanent until end of combat) **3B:** Adds secondary ``
Adrenaline ``	**1A:** Add **1B:** Add	**2A (from 1A):** Target becomes ALLY_SINGLE **2B (from 1B):** Target becomes ALLY_SINGLE	**3A:** Target becomes ALLY_ALL **3B:** Base Energy restore increased to 5
Poison Vial ``	**1A:** Target becomes ENEMY_ALL **1B:** Add `` (Weakness)	**2A (from 1A):** Increase Dur to 6 **2B (from 1B):** Add `` (Lethargy)	**3A:** Adds `` (Current HP massive percent cut) **3B:** Cost reduced to 0
Meditate ``	**1A:** Add **1B:** Add	**2A (from 1A):** Target becomes ALLY_SINGLE **2B (from 1B):**	**3A:** Add `` (Action Surge) **3B:** Automatically purges all negative

		Energy restore increased to 3	modifiers with duration > 0

In the *Meditate* example, Tier 3A provides incredible turn-order dominance by spiking Speed, while Tier 3B provides unparalleled hazard and debuff mitigation. Neither choice is strictly optimal in a vacuum; the "correct" choice is entirely dependent on the specific procedural seed the player is currently navigating.

IV. Procedural Generation Pipeline for Campaign Maps

Campaign structure strictly enforces a 6-region macro-graph, with each region comprising exactly three narrative micro-nodes: Approach, Settlement, and Stronghold.[11] Utilizing a bucket-distribution algorithm adapted from constrained spatial partitioning concepts found in successful roguelites [23], the procedural generation ensures structured pacing while retaining high variance in encounter types.

Macro and Micro Constraints

- **Macro-Map:** The player begins at Region 1 (Outpost) and must conquer an interconnected web to reach Region 6. The web consists of branching difficulty paths (e.g., clearing Region 1 unlocks Region 2 and 3, allowing non-linear progression to the center).
- **Micro-Map (The Region):** Upon selecting a Region, the player progresses linearly through Node 1 (Approach) $\rightarrow$ Node 2 (Settlement) $\rightarrow$ Node 3 (Stronghold).[26]
- **Bucket System Distribution:** Encounters are drawn from weighted buckets based on the node's narrative type. An "Approach" favors environmental storytelling via Hazards (60%) or Events (40%). A "Settlement" favors standard Combat (70%) or complex Events (30%). A "Stronghold" is strictly an Elite Combat encounter (100%), terminating the region.

Procedural Pipeline Pseudocode

The following Python logic demonstrates the constraint solver that builds the campaign map while respecting the node hierarchy and narrative structure.[28]

Python

```python
import random

class EncounterDef:
    def __init__(self, id, enc_type, tier, data):
```

```
        self.id = id
        self.type = enc_type # "COMBAT", "COMBAT_ELITE", "HAZARD", "EVENT"
        self.tier = tier # 1 through 4
        self.data = data # JSON modifier arrays and entity stats

class CampaignGenerator:
    def __init__(self, seed, encounters_json):
        random.seed(seed)
        self.encounters_db = self._load_encounters(encounters_json)

    def _load_encounters(self, json_data):
        # Parses JSON into EncounterDef objects
        return

    def generate_campaign(self):
        campaign_graph = {}
        # Define 6 regions with branching difficulty tiers
        regions =

        for region in regions:
            campaign_graph[region["id"]] = {
                "name": region["name"],
                "tier": region["tier"],
                "nodes": self._generate_region_micro_nodes(region["tier"])
            }

        return campaign_graph

    def _generate_region_micro_nodes(self, tier):
        # Enforce narrative arc: Approach -> Settlement -> Stronghold
        nodes =

        # 1. Approach Generation (Weight: Hazard 60%, Event 40%)
        approach_pool = * 60 + * 40
        nodes.append({
            "stage": "Approach",
            "encounter": self._pull_from_bucket(random.choice(approach_pool), tier)
        })

        # 2. Settlement Generation (Weight: Combat 70%, Event 30%)
        settlement_pool = * 70 + * 30
        nodes.append({
            "stage": "Settlement",
```

```
        "encounter": self._pull_from_bucket(random.choice(settlement_pool), tier)
    })

    # 3. Stronghold Generation (Weight: Elite Combat 100%)
    nodes.append({
        "stage": "Stronghold",
        "encounter": self._pull_from_bucket("COMBAT_ELITE", tier)
    })

    return nodes

def _pull_from_bucket(self, encounter_type, tier):
    # Filters database for encounters matching the type and difficulty tier
    valid_encounters = [e for e in self.encounters_db
                        if e.type == encounter_type and e.tier == tier]
    if not valid_encounters:
        # Fallback to lower tier if pool is exhausted to prevent crashing
        valid_encounters = [e for e in self.encounters_db
                            if e.type == encounter_type and e.tier == max(1, tier-1)]

    return random.choice(valid_encounters).id
```

By separating the generation into defined layers, the game guarantees that every session maintains a recognizable escalation of threat. Information is revealed in stages: players initially see only the Region modifiers. Researching a region uncovers the Approach type. Further research uncovers the Stronghold identity, allowing players to tailor their strategic choices.

V. World Deck and Strategic Trade-Offs

Transitioning between regions, the player interacts with the "World Deck"—a strategic layer determining meta-modifications for the subsequent region.[30] Every card in the World Deck strictly enforces a "Trade-Off" design philosophy.[31] There are no strictly positive choices; player agency is derived from minimizing mechanical risks while maximizing contextual benefits based on their current roster and the upcoming region's profile.

When reviewing trade-offs, a seemingly minor downside can result in a cascading failure if misaligned with the upcoming encounter type. Conversely, severe downsides can be entirely mitigated by a synergizing Outpost upgrade or specific character passive.[34]

Table 5: World Deck Trade-Off Cards

World Card Name	Upside Modifier	Downside	Contextual

	(Benefit)	Modifier (Cost)	Strategy and Analysis
Forced March	+30% Speed (All Allies)	-20% HP (All Allies)	High value if the upcoming Stronghold features low-HP, high-Power enemies requiring turn-1 alpha-strikes. Fatal against attrition/hazard encounters.
Rations Cut	+2 Energy base max	-25% Power (All Allies)	Ideal for utility/defense-heavy rosters that rely on playing multiple weak debuffs rather than raw flat damage output.
Reckless Assault	+40% Power (All Allies)	-40% Defense (All Allies)	Strong when paired with characters possessing innate Speed passives to kill enemies before taking damage. Disastrous against swarms.
Heavy Armor	+30 Defense (All Allies)	-30% Speed (All Allies)	Devastating downside against fast enemies, but trivializes regions dominated by multiple low-Power, multi-hit swarms.
Blood Magic	Base card costs -1 Energy	Flat -8 HP per card played	Extremely dangerous in long battles. Combines

			degenerately with "Heal on cast" card upgrades if not balanced carefully.
Fog of War	Enemies start with -50% Pwr	Player roster starts with Blind	Reduces burst threat, but prevents the player from executing precise targeting combinations in round 1.
Overclocked	+60% Speed (All Allies)	Energy does not refresh Turn 2	Creates an "all-in" dependency on Turn 1. If enemies survive to Turn 2, the player effectively skips an entire round of output.
Vampiric Contract	+15 HP heal on enemy kill	Base max HP -30%	Excellent for multi-enemy Settlement nodes; mathematically poor against single-target Stronghold bosses where no kills occur until the fight ends.
Scavenger's Greed	Reward output +50%	Stronghold Elite gets +50% Pwr	A standard greed mechanic. Picks are obvious if the player is snowballing, but run-ending if struggling.

Hyper-Metabolism	+50% HP (All Allies)	Status effect duration x2	Grants massive survivability but makes Hazard encounters (which rely on damage-over-time effects) mathematically fatal.
Glass Cannon	+100% Power (All Allies)	HP permanently set to 1	The ultimate risk. Synergizes perfectly with extreme Speed builds and Defense/Shield spam, but instantly loses to pre-emptive Hazards.
Pacifism Protocol	+50 Defense (All Allies)	Cannot play Attack cards Turn 1	Excellent for setting up buffs, useless for aggro builds.
Leyline Tap	Energy refreshes to Max + 5	Lose 10% Current HP per turn	High-octane attrition mode. Forces the player to end fights in 3 turns or die to self-inflicted damage.
Tunnel Vision	+50% Power vs Elites	-50% Power vs Minions	Highly specific pick based on whether the region is a Boss focus or a swarm focus.
Unstable Mutagen	Random Stat +50%	Random Stat -50%	True RNG. Often skipped unless the

			player has earned a skip token, acting as a "trap" card for desperate runs.
Barricaded	Start with +100 Defense	Lose 20 Defense per turn	Incredible for surviving immediate ambushes, but falls apart in prolonged battles.
Cursed Relic	+1 Character drafted early	All Enemies gain +15% Stats	Accelerates incremental power growth at the severe cost of immediate difficulty scaling.
Martyrdom	Lowest HP Ally gets +200% Def	Highest HP Ally gets -50% HP	Requires precise roster management to manipulate who receives the buff and who takes the penalty.
Temporal Shift	+100% Speed on Turn 1	-50% Speed on Turn 2, 3, 4	Ensures first strike capability but guarantees the enemy will take multiple consecutive turns immediately after.
Echo Chamber	Played cards trigger twice	Energy cost of all cards x2	Shifts the deck paradigm from playing many cheap cards to playing one incredibly powerful, expensive card per turn.

By ensuring that every option has a mathematical consequence mapped to the 5-stat model, the game eliminates "auto-pick" scenarios. Evaluating these cards becomes an exercise in reading the procedural map and measuring the specific statistical margins of the active roster.

VI. React Frontend State Architecture

For a browser-based, client-side application devoid of an underlying native engine (e.g., Unity or Godot), the frontend architecture relies on a deterministic, unidirectional data flow reminiscent of Redux.[16] Because turn-based games do not require real-time physics calculations, the game state acts as a pure mathematical ledger.

Redux-Style State Machine and Serialization

The game operates as a finite state machine transitioning strictly between distinct phases: WORLD_MAP, ENCOUNTER_ACTIVE, and OUTPOST_UPGRADE. The entire game state is stored in a singular, serializable JSON blob.[38] This structure guarantees that at any point, the game can be stringified and saved to localStorage or a remote database without losing combat flow or encountering desynchronization.

State Shape Definition:

TypeScript

```
interface GameState {
  run_seed: string;
  current_phase: 'WORLD_MAP' | 'COMBAT' | 'EVENT' | 'OUTPOST';
  active_roster: Character;
  inventory_deck: Card;
  world_modifiers: Modifier;
  encounter_state: {
    turn_queue: string; // Entity IDs sorted by CT
    enemies: Entity;
    combat_log: ActionTuple;
    active_entity_id: string | null;
  }
}
```

Undo/Redo Capability via Memento/Command Pattern

Because unwinnable seeds and punitive math are core to the design, players will inevitably

make interface misclicks. Implementing an Undo system in React turn-based engines requires the Memento or Command pattern.[40]

Instead of attempting to mathematically reverse operations (which fails spectacularly due to floating-point truncation in percentage modifiers), the engine utilizes a state-history array.[43] Each time a card is played and the ResolverEngine completes its tick, a deep copy of the GameState is pushed to a history array. An undo() function simply pops the current state and restores history[history.length - 1].

Component Tree Architecture

The UI decoupled from the simulation engine. The React components simply ingest the GameState and map it to visual representations. Hotkey bindings (1-6) trigger dispatch actions that send the index of the selected card to the reducer.

JavaScript

```
<App>
 <KeyboardController hotkeys={{ '1-6': playCardIndex, 'Z': undoLastAction }}>
  <StateRouter phase={current_phase}>

   {/* Renders the procedural map and Trade-off cards */}
   <Phase_WorldMap>
     <WorldDeckSelector cards={drawn_world_cards} onSelect={dispatchWorldChoice} />
     <CampaignGraph nodes={procedural_nodes} activeNode={current_node} />
   </Phase_WorldMap>

   {/* Renders the Combat Engine view */}
   <Phase_Encounter>
     <CombatArena>
       <EntityRenderer type="ENEMY" data={encounter_state.enemies} />
       <EntityRenderer type="PARTY" data={active_roster} />
     </CombatArena>

     {/* Shared card interface across all interactive screens */}
     <HandController>
       {active_hand.map((card, index) => (
         <CardComponent
           key={card.id}
           data={card}
           hotkey={index + 1}
           onClick={() => dispatchCardPlay(card)}
         />
```

```
        ))}
      </HandController>
    </Phase_Encounter>

    {/* Renders Meta-progression */}
    <Phase_Outpost>
      <RosterManagement characters={active_roster} />
      <UpgradeAltar deck={inventory_deck} onUpgrade={dispatchUpgrade} />
    </Phase_Outpost>

  </StateRouter>
 </KeyboardController>
</App>
```

VII. Monte Carlo Balance Simulation Framework

The intention of the procedural generator is *not* to ensure every seed is winnable; unwinnable seeds are a core, intentional feature designed to force failure and subsequent incremental meta-progression. Therefore, traditional seed validation logic is obsolete. Instead, the objective of the Python backend is to conduct massive automated playtests via Monte Carlo simulation to validate that the *distribution* of win rates falls within healthy design margins (e.g., a 40-70% win rate across 10,000 seeds).[45]

Furthermore, the simulation seeks to identify "degenerate" combinations [47]—instances where a specific combination of modifiers breaks the mathematical boundaries of the 5-stat model, bypassing intended strategic friction entirely.[48]

Simulation Goals and Degeneracy Metrics

1. **Win-Rate Distribution:** Executing tens of thousands of random seeds using distinct AI heuristics (Aggressive, Defensive, Balanced) to chart the success bell curve.
2. **Degenerate Combo Detection:** Identifying modifier combinations that result in a 100% win rate against Stronghold encounters regardless of the seed. This uses concepts derived from Nash equilibrium support enumeration to find outlier dominant strategies.[47]
3. **Upgrade Path Dominance:** Tracking if the simulation heavily favors Tier 3A over Tier 3B across successful runs. If an upgrade is chosen >85% of the time in winning runs, it indicates a failure in "meaningful choice" design.
4. **Trade-Off Analysis:** Checking if certain World Deck cards result in statistically insignificant differences in win rate, meaning the downside is effectively a non-issue and the trade-off is broken.

Python Monte Carlo Harness Outline

Python

```python
import copy
import random
from statistics import mean, stdev
from collections import Counter

class MonteCarloSimulator:
    def __init__(self, iterations=10000):
        self.iterations = iterations
        self.results =
        self.upgrade_selections =

    def run_simulations(self):
        for i in range(self.iterations):
            seed = i
            campaign = CampaignGenerator(seed, "encounters.json").generate_campaign()

            # AI Agents programmed with distinct evaluation heuristics
            agent = random.choice()

            result = self._simulate_run(campaign, agent, seed)
            self.results.append(result)

        self._analyze_metrics()

    def _simulate_run(self, campaign, agent, seed):
        roster = self._generate_starting_roster()
        deck = self._generate_starting_deck()
        state = {"roster": roster, "deck": deck, "regions_cleared": 0}

        for region_id, data in campaign.items():
            for node in data["nodes"]:
                engine = ResolverEngine(state["roster"], node["encounter"]["enemies"])

                # The agent evaluates the GameState and plays cards
                outcome = agent.play_encounter(engine)

                if outcome == "LOSS":
                    return {
```

```
                "seed": seed,
                "status": "LOSS",
                "progress": state["regions_cleared"],
                "final_deck": [c.id for c in state["deck"]]
            }

        state["regions_cleared"] += 1

        # Record AI upgrade selections to check for path dominance
        upgraded_card = agent.select_card_upgrade(state["deck"])
        self.upgrade_selections.append(upgraded_card.upgrade_path)

        # Apply AI selection for world deck trade-off
        world_card = agent.choose_world_card()
        self._apply_world_card(state, world_card)

    return {
        "seed": seed,
        "status": "WIN",
        "progress": 6,
        "agent": agent.__class__.__name__,
        "final_deck": [c.id for c in state["deck"]]
    }

def _analyze_metrics(self):
    wins = [r for r in self.results if r["status"] == "WIN"]
    win_rate = len(wins) / self.iterations
    print(f"Overall Win Rate: {win_rate * 100:.2f}%")

    if not (0.40 <= win_rate <= 0.70):
        print("WARNING: Win rate distribution falls outside healthy 40-70% margin.")

    # Detect Upgrade Dominance
    upgrade_counts = Counter(self.upgrade_selections)
    for path, count in upgrade_counts.items():
        pick_rate = count / len(self.upgrade_selections)
        if pick_rate > 0.85:
            print(f"DEGENERACY DETECTED: Upgrade Path {path} dominates ({pick_rate*100:.1f}%)")

    # Detect Degenerate Combos
    # A degenerate strategy is flagged if a specific card combination
    # appears in >95% of winning seeds but <50% of losing seeds.
    winning_decks = [tuple(sorted(w["final_deck"])) for w in wins]
```

```
    deck_counts = Counter(winning_decks)
    # Further statistical correlation logic goes here...
```

VIII. Pitfall Avoidance Checklist

Hybridizing incremental progression with a finite roguelite structure inherently risks systemic collisions. When attempting to marry constrained decision-making with procedural generation, several well-documented pitfalls emerge, particularly surrounding action economy and statistical inflation.[52]

1. **The "Speed" Stat Action-Economy Collapse**
   - **The Pitfall:** In turn-based systems governed by continuous CT thresholds, percentage modifiers to Speed scale exponentially in efficiency. A +100% Speed modifier effectively allows a character to lap an enemy, acting as a permanent Stun alongside a 2x damage multiplier.
   - **Mitigation:** Speed must be the only stat heavily restricted in percentage modifiers. Alternatively, introduce "Fatigue" modifiers (flat energy cost increases) that apply to a character when multiple turns are taken within a single enemy CT cycle, forcing an economic hard cap on infinite speed loops.
2. **Trade-Off Nullification via Incremental Upgrades**
   - **The Pitfall:** A World Card granting -20% HP as a downside becomes entirely irrelevant if an Outpost Upgrade grants a flat +500 HP at the start of a run. The incremental meta-scaling breaks the strategic tension of the roguelite phase, reducing trade-offs to non-decisions.[55]
   - **Mitigation:** The universal modifier system must strictly enforce that Meta-progression (Outpost Upgrades) applies to *Base Stats*, whereas Region and World Card modifiers apply to *Current Stats* post-calculation. Ensure steep numerical ceilings for incremental progression to retain the fatal threat of percentage-based downsides.
3. **Economy Manipulation Dominance**
   - **The Pitfall:** When upgrading cards, adding a "Draw 1 card" or "Energy Cost -1" effect will statistically dominate any alternative upgrade offering flat damage increases.[56] Card advantage and resource generation are universally superior to flat output.
   - **Mitigation:** Economy manipulation must never be paired directly against mathematical output in an upgrade branch. If Tier 1A grants Energy reduction, Tier 1B must offer a comparable mechanical subversion (e.g., adding an AoE tag or bypassing Defense entirely).
4. **UI State Desynchronization**
   - **The Pitfall:** Relying on React's asynchronous state updates (setState) inside loops calculating combat logic leads to visual desynchronization, where the UI renders frame $X + 2$ while the engine calculates frame X.
   - **Mitigation:** The ResolverEngine must operate entirely independently of the React component tree. The engine calculates the entire turn instantly and outputs an array

of ActionTuples representing discrete events. The React UI then acts as a "dumb renderer," consuming the tuples sequentially with forced delays (setTimeout or CSS transition end events) to display the outcome to the player, maintaining strict separation between mathematical state and visual rendering.

5. **Infinite Duration Stacking (The Armor Loop)**
   - **The Pitfall:** If Defense FLAT_ADD cards have a duration of 3 turns, but a character's Speed allows them to play the card every 2 turns, Defense stacks infinitely until the character is mathematically invulnerable.
   - **Mitigation:** The engine must enforce explicit stacking rules within the JSON definitions. A modifier must either replace existing identical modifiers (refreshing the duration) or be tagged as STACKABLE. Restricting stackability on defensive parameters forces players to time their mitigations rather than blindly spamming them.

Works cited

1. Why and when to use Json files to make / modify levels : r/gamedev - Reddit, accessed March 2, 2026, https://www.reddit.com/r/gamedev/comments/1rf5m9j/why_and_when_to_use_json_files_to_make_modify/
2. Exploring Game Saves: JSON # 2 Arrays | Game Dev Tutorial - YouTube, accessed March 2, 2026, https://www.youtube.com/watch?v=CrT4RXyYBxs
3. Full wiki export - LegendKeeper 0.7.4, accessed March 2, 2026, https://www.legendkeeper.com/changelog/legendkeeper-0-7-4-full-wiki-export/
4. Save system for complex, data-heavy game? : r/godot - Reddit, accessed March 2, 2026, https://www.reddit.com/r/godot/comments/1g76vku/save_system_for_complex_dataheavy_game/
5. Steam Workshop::[OLD] Dukko's 3.7 - Stellaris, accessed March 2, 2026, https://steamcommunity.com/sharedfiles/filedetails/?id=2947106339
6. Niko's Guide to the Modded Stellaris - Steam Community, accessed March 2, 2026, https://steamcommunity.com/sharedfiles/filedetails/?id=2875768706
7. LegendKeeper 0.16.1.0, accessed March 2, 2026, https://www.legendkeeper.com/changelog/legendkeeper-0-16-1-0/
8. How to create data driven effects/abilities for collectable card game, accessed March 2, 2026, https://gamedev.stackexchange.com/questions/130642/how-to-create-data-driven-effects-abilities-for-collectable-card-game
9. Dynamic JSON Modding System - load an in game front end for any JSON shape. Let me know if you'd be interested in me releasing this on the Asset Store. : r/gamedev - Reddit, accessed March 2, 2026, https://www.reddit.com/r/gamedev/comments/uqgwxl/dynamic_json_modding_system_load_an_in_game_front/
10. turn-based combat : r/gamedev - Reddit, accessed March 2, 2026, https://www.reddit.com/r/gamedev/comments/12ksx0l/turnbased_combat/

11. Building Lord of The Rings-Inspired Dwarven Mountain Stronghold in 3D - 80 Level, accessed March 2, 2026, https://80.lv/articles/building-lord-of-the-rings-inspired-dwarven-mountain-stronghold-in-3d
12. Impressions of the GDMC AI Settlement Generation Challenge in Minecraft - arXiv.org, accessed March 2, 2026, https://arxiv.org/pdf/2108.02955
13. Experts and Gamers on Immersion into Reconstructed Strongholds - ResearchGate, accessed March 2, 2026, https://www.researchgate.net/publication/346513782_Experts_and_Gamers_on_Immersion_into_Reconstructed_Strongholds
14. What's a flexible way to handle combat events in a turn based battle system? - Reddit, accessed March 2, 2026, https://www.reddit.com/r/gamedev/comments/eur6y3/whats_a_flexible_way_to_handle_combat_events_in_a/
15. Turn-Based Combat - Speed & Turn Order : r/gamedesign - Reddit, accessed March 2, 2026, https://www.reddit.com/r/gamedesign/comments/4bezuv/turnbased_combat_speed_turn_order/
16. How to design a state flow for a react/redux flashcard game? - Stack Overflow, accessed March 2, 2026, https://stackoverflow.com/questions/35220169/how-to-design-a-state-flow-for-a-react-redux-flashcard-game
17. Architecting a turn based game engine with Redux | trash moon, accessed March 2, 2026, https://trashmoon.com/blog/2019/architecting-a-turn-based-game-engine-with-redux/
18. Turn Based Strategy Games | Implementing Card Effects (P3) - YouTube, accessed March 2, 2026, https://www.youtube.com/watch?v=PIY_6I8Fzpw
19. Cards | Across the Obelisk Wiki | Fandom, accessed March 2, 2026, https://ato.fandom.com/wiki/Cards
20. Deckbuilders: The Best Today, Top Demos, Coming Soon 12/25 | Page 2, accessed March 2, 2026, https://forums.pcgamer.com/threads/deckbuilders-the-best-today-top-demos-coming-soon-12-25.149833/page-2
21. Why does Slay the Spire still dominate deckbuilders? : r/deckbuildingroguelike - Reddit, accessed March 2, 2026, https://www.reddit.com/r/deckbuildingroguelike/comments/1q1nhnw/why_does_slay_the_spire_still_dominate/
22. Across the Obelisk - Steam Community, accessed March 2, 2026, https://steamcommunity.com/app/1385380/positivereviews/?l=english&browsefilter=toprated
23. I have reverse-engineered map generation algorithm from Slay the ..., accessed March 2, 2026, https://www.reddit.com/r/slaythespire/comments/ndqweh/i_have_reverseengineered_map_generation_algorithm/

24. Procedural Map Generation inspired by Slay the Spire : r/gamedev - Reddit, accessed March 2, 2026, https://www.reddit.com/r/gamedev/comments/fedcvu/procedural_map_generation_inspired_by_slay_the/
25. In-Depth (~2 Hours) Roguelike Map Generation (Slay the Spire-style) : r/godot - Reddit, accessed March 2, 2026, https://www.reddit.com/r/godot/comments/1awbg1j/indepth_2_hours_roguelike_map_generation_slay_the/
26. Prompt-Driven Procedural Content Generation for Virtual Worlds Using Consumer-Level Large Language Models - ScholarWorks, accessed March 2, 2026, https://scholarworks.calstate.edu/downloads/0z7095805
27. Master Thesis Project Procedural Narrative Generation Through Emotionally Interesting Non-Player Characters, accessed March 2, 2026, https://lnu.diva-portal.org/smash/get/diva2:1231584/FULLTEXT01.pdf
28. Algorithms used for procedurally generated dungeons - Diva-portal.org, accessed March 2, 2026, http://www.diva-portal.org/smash/get/diva2:1980492/FULLTEXT01.pdf
29. Procedural Content Generation, accessed March 2, 2026, https://faculty.cc.gatech.edu/~surban6/2018sp-gameAI/lectures/2018_04_05-ProceduralContentGeneration_concluded.pdf
30. Card Mechanics - Official Star Traders Wiki, accessed March 2, 2026, https://startraders.fandom.com/wiki/Card_Mechanics
31. Trade-Offs, Dilemmas, Sacrifices, Risk & Reward | by Myk Eff | Understanding Games, accessed March 2, 2026, https://medium.com/understanding-games/trade-offs-dilemmas-sacrifices-risk-reward-46acd014e435
32. Game design where you can tell it wasn't properly tested or rushed? : r/TwoBestFriendsPlay, accessed March 2, 2026, https://www.reddit.com/r/TwoBestFriendsPlay/comments/1rftvbh/game_design_where_you_can_tell_it_wasnt_properly/
33. The Broken Design of Rogue. A case study in trade-offs : r/hearthstone - Reddit, accessed March 2, 2026, https://www.reddit.com/r/hearthstone/comments/10ct125/the_broken_design_of_rogue_a_case_study_in/
34. Despotism 3k - App Store, accessed March 2, 2026, https://apps.apple.com/tr/app/despotism-3k/id1462908562
35. 60+ Games Like Infectonator 3: Apocalypse - MiniReview, accessed March 2, 2026, https://minireview.io/simulation/infectonator-3-apocalypse/games-like
36. I wanna play the bad guy : r/gamingsuggestions - Reddit, accessed March 2, 2026, https://www.reddit.com/r/gamingsuggestions/comments/1pcmc1q/i_wanna_play_the_bad_guy/
37. This engine uses a Redux-like architecture. You have a State type (containing da... | Hacker News, accessed March 2, 2026, https://news.ycombinator.com/item?id=42471288
38. Serializing and saving game state each turn best practice? : r/gamedev - Reddit,

accessed March 2, 2026, https://www.reddit.com/r/gamedev/comments/1onbpat/serializing_and_saving_game_state_each_turn_best/
39. “How I Made a Turn Based Game in React” Part 1: Basic Movement - Medium, accessed March 2, 2026, https://medium.com/@drewisatlas/how-i-made-a-turn-based-game-in-react-part-1-basic-movement-6d892cb93947
40. Serialization-Based Undo, accessed March 2, 2026, https://prideout.net/blog/undo_system/
41. How to proper implement Undo Redo system in React using context and command pattern?, accessed March 2, 2026, https://stackoverflow.com/questions/75525403/how-to-proper-implement-undo-redo-system-in-react-using-context-and-command-patt
42. You Don't Know Undo/Redo - DEV Community, accessed March 2, 2026, https://dev.to/isaachagoel/you-dont-know-undoredo-4hol
43. Build a Reusable Undo/Redo State Manager in React (with Persistence Support) - Medium, accessed March 2, 2026, https://medium.com/@kom50/build-a-reusable-undo-redo-state-manager-in-react-with-persistence-support-ef91e7792d66
44. How do I go about creating an Undo system in a grid based game? - Reddit, accessed March 2, 2026, https://www.reddit.com/r/gamedev/comments/1gwdrb3/how_do_i_go_about_creating_an_undo_system_in_a/
45. Monte Carlo Simulation: Random Sampling, Trading and Python - QuantInsti Blog, accessed March 2, 2026, https://blog.quantinsti.com/monte-carlo-simulation/
46. Monte Carlo Simulation with Python, accessed March 2, 2026, https://pbpython.com/monte-carlo.html
47. Degenerate games — Nashpy 0.0.36 documentation, accessed March 2, 2026, https://nashpy.readthedocs.io/en/v0.0.36/text-book/degenerate-games.html
48. Using Python to check if strategies are best responses to each other - YouTube, accessed March 2, 2026, https://www.youtube.com/watch?v=h6zSP8G1aQw
49. Handle Degenerate games — Nashpy 0.0.43 documentation, accessed March 2, 2026, https://nashpy.readthedocs.io/en/stable/how-to/handle-degenerate-games.html
50. EGTtools: Evolutionary game dynamics in Python - PMC, accessed March 2, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10123342/
51. Game Theory concepts with application in Python using Nashpy (Part 1), accessed March 2, 2026, https://towardsdatascience.com/game-theory-in-python-with-nashpy-cb5dceab262c/
52. Big development pitfalls or no-no's? Share your experience as a dev! : r/incremental_games, accessed March 2, 2026, https://www.reddit.com/r/incremental_games/comments/1mwrfeo/big_development_pitfalls_or_nonos_share_your/
53. Roguelikes, persistency, and progression | Game Maker's Toolkit - Reddit,

accessed March 2, 2026, https://www.reddit.com/r/Games/comments/ako8s2/roguelikes_persistency_and_progression_game/
54. Recognize and Avoid These Game Development Mistakes | by Jay Zeek - Medium, accessed March 2, 2026, https://medium.com/super-jump/recognize-and-avoid-these-game-development-mistakes-c001b6e988fd
55. Meta progression in roguelites was fun for a while, but it's starting to feel unrewarding, accessed March 2, 2026, https://www.reddit.com/r/truevideogames/comments/1oj0bdj/meta_progression_in_roguelites_was_fun_for_a/
56. Defect A20H Tier list by 68% player : r/slaythespire - Reddit, accessed March 2, 2026, https://www.reddit.com/r/slaythespire/comments/1e6qx0m/defect_a20h_tier_list_by_68_player/